Day 13 — GitHub Integration

Status: ■ Not started

Objectives

- Connect Jenkins to a GitHub repository securely
- Configure webhooks for push-triggered builds
- Set up a Multibranch Pipeline that auto-discovers branches and PRs
- Understand GitHub credential types (PAT vs SSH key vs GitHub App)

Lab Steps

Create GitHub credentials in Jenkins

- Generate a fine-scoped Personal Access Token (repo scope)
- Add as "Username with password" or "Secret text" credential

Configure a webhook

- GitHub repo → Settings → Webhooks → Add webhook
- Payload URL: `http://<jenkins-host>/github-webhook/`
- Content type: `application/json`, event: `push`

Create a Multibranch Pipeline

- New Item → Multibranch Pipeline
- Branch source: GitHub, point at your repo, use the PAT credential
- Build configuration: by Jenkinsfile

Test the flow

- Push a commit to a feature branch
- Confirm Jenkins auto-discovers the branch and runs the Jenkinsfile
- Open a PR and confirm PR-triggered builds work

(Optional) GitHub status checks

- Enable commit status publishing so PRs show build pass/fail directly in GitHub

Key Takeaways

- Webhooks are strongly preferred over polling — near-instant, lower load
- Multibranch pipelines remove the need to manually create a job per branch
- Use least-privilege tokens; rotate them periodically

Checklist

- [ ] GitHub credentials configured in Jenkins
- [ ] Webhook delivering successfully (check GitHub's "Recent Deliveries")
- [ ] Multibranch Pipeline auto-building branches
- [ ] PR builds triggering and reporting status back to GitHub

Day 13 — Interview Questions: GitHub Integration

Why is a webhook generally preferred over SCM polling?

Walk through what happens end-to-end when a developer pushes a commit, from GitHub to a running build.

What credential types work with GitHub, and when would you choose each (PAT vs SSH key vs GitHub App)?

What is a Multibranch Pipeline, and how does it differ from creating one job per branch manually?

How does Jenkins report build status back to a GitHub PR, and why does that matter for branch protection rules?

What's the difference between building "the PR head only" vs "the PR merged with target branch"? Which would you choose and why?

How would you debug a webhook that isn't triggering a build?

What security risks come with granting a Jenkins credential broad GitHub scopes, and how do you mitigate them?

<details><summary>Answer notes</summary>

• Q1: Webhooks are push-based (near-instant, low overhead); polling adds latency and unnecessary API/Git load at scale.

• Q6: "Merged with target" simulates the actual post-merge state, catching integration issues polling the source branch alone would miss — generally the safer default for PR gating.

• Q7: Check GitHub's webhook "Recent Deliveries" for response codes, verify the payload URL and that the job has the GitHub trigger enabled, and check firewall/reachability from GitHub's IPs.

</details>